JavaScript master doc

Variables

Déclaration

3 mots clé :

- `let` : bloc-scoped (déclaration d'une variable dont la portée est celle du bloc courant)
- `const` :bloc-scoped (déclaration d'une constante dont la portée est celle du bloc courant)
- `var` : function-scoped (déclaration d'une variable dont la portée est celle de la fonction courante)

```
let maVariable = valeur;
```

Nommage des variables :

- pas de mot clé
- lettres, chiffres, $ et _ uniquement
- pas commencer par un chiffre
- favoriser le camelCase
- nom pertinent

```
const MA_CONSTANTE = valeur;
```

Nommage des constantes :

- valeur hardcoded ⇒ majuscules
- valeur calculée ⇒ comme pour les variables
- systématiquement `const` pour des tableaux et objets

`var` : permet de créer une variable accessible en dehors du bloc où elle a été créée
⚠ quand même function-scoped, elle est détruite à la fin de la fonction
⇒ risque de se mélanger avec les variables, utiliser `let` ou `const`

aussi possible de ne pas utiliser de mot clé pour déclarer une variable
⇒ utilise `var`, donc mêmes problèmes
⇒ utiliser `use strict` au début du fichier pour empêcher ce comportement (sinon risque de créer une nouvelle variable avec une faute de frappe !)

en JavaScript, les variables ne sont pas typées ⇒ possible de changer le type
TypeScript : subset du JavaScript qui type les variables

Types primitifs

- Number
- Boolean

- String
- null
- undefined

Number

un seul type pour tous les nombres

valeurs numériques spéciales :

- Infinity -> si on divise par 0
- NaN -> valeur numérique pour indiquer que ce n'est pas un nombre

Boolean

vrai ou faux

String

guillemets simples ou doubles, choisir un style et s'y tenir

opérateur binaire `+` : entre deux Strings ou entre un String et un autre type, concatène

```
"ceci" + "cela" // "cecicela"
"mot" + 3 // "mot3"
```

opérateur unaire `+` : devant autre chose qu'un nombre, transforme en nombre

```
+"10" // 10
3 + +"6" // 9
+"Anna" // NaN
+false // 0
```

les Strings sont immutables (on ne peut pas en changer les parties), mais on a des méthodes pour travailler avec (sans le modifier pour autant)

les backticks (accents graves) créent des template strings qui permettent d'intégrer des expressions ou des variables directement dans la chaîne de caractères avec `${...}`

```
let nom = "Ada";

`Hello, ${nom}!`; // Hello, Ada!

`Le résultat est ${1 + 2}` // Le résultat est 3

`Random Number : ${Math.random * 100}` // Random Number : 39
```

Null

l'absence intentionnelle de valeur, doit être assignée

Undefined

type d'une variable qui n'a pas de valeur assignée

Comparaisons

double égal vérifie l'égalité mais pas le type ⇒ peut produire des résultats inattendus
triple égal vérifie l'égalité et le type ⇒ utilisé dans 99% des cas

```
"1" == 1 // true
"1" === 1 // false

0 == false // true
0 === false // false
```

comparaisons utilisées pour créer des embranchements dans le programme avec `if`, `else` et `else if`

Valeur booléenne des expressions

""	**false**
null	false
undefined	false
NaN	false
0	false
autre	true

opérateur de coalescence des nuls

retourne la valeur de la variable si elle existe ou une valeur prédéfinie si la variable est de type null ou undefined

```
let user;
console.log(user ?? "anonymous"); // Anonymous

let user = "Bruce";
console.log(user ?? "anonymous"); // Bruce
```

Ternaire

vérifie la condition et retourne la première partie si vrai et la deuxième si faux

```
condition ? "if true" : "if false";
```

Switch

pour comparer une même variable avec plusieurs valeurs possibles

```
switch(variable) {
    case 0: instructions; break;
    case 1: instructions; break;
    case 2: instructions; break;
    default: instructions;
}
```

Tableaux

créés avec `[]`
peuvent mélanger les types :

```
let tab = [];
let mixed = [true, null, 42, "chat", NaN]
```

Méthodes basiques

- push : ajoute à la fin
- pop : supprime de la fin
- shift : enleve du début
- unshift : ajoute au début
- concat : fusionne des tableaux
- includes : cherche une valeur
- indexOf : obtient l'index d'une valeur
- join : crée un String à partir du tableau
- reverse : inverse un tableau
- slice : copie une portion du tableau
- splice : enlève/remplace des éléments
- sort : classe le tableau selon les codes Unicode (modifie le tableau original)
- toSorted : comme sort, mais sans modifier le tableau original

Objets

créés avec `{}` (pas besoin de constructeur)
sont en fait une collection de propriétés sous la forme de paires "clé: valeur"

```
const user = {}; // objet vide

const user = {
    firstName: "Merlin",
    lastName: "Emrys",
```

```
    age: 21,
    function: servant
}

user.age; // 21

user.isMagical = true; // ajoute la propriété isMagical: true

delete user.function;
```

Boucles

for

```
for (let i = 0; i < 3; i++) {
    instructions
}
```

while

```
while (condition) {
    instructions
}
```

sur un tableau ou autre itérable

```
for (item of iterable) {
    instructions;
}
```

Fonctions

Déclaration

```
function nomFonction(param) {
    //instructions
    return param;
}
```

une fonction retourne toujours quelque chose ; si l'instruction `return` n'est pas donnée, le résultat est `undefined`

une fonction peut être appelée avant sa déclaration ⇒ hoisting

```
console.log(add(3, 7)); // affiche 10

function add(a, b) {
    return a + b;
```

```
}

console.log(add(3, 7)); // affiche 10
```

une fonction peut être passée en paramètre d'une autre fonction, passée en retour, attribuée à une variable, ou encore stockée dans un tableau ou un objet

Expression de fonction

possible de stocker une fonction dans une variable (expression de fonction)
⇒ ces expressions ne supportent pas le hoisting

```
console.log(add(3, 7)); // affiche une erreur

const add = function(a, b) {
    return a + b;
}

console.log(add(3, 7)); // affiche 10
```

Fonction fléchée

```
const nomFonction = param => resultat;

// par ex :

const add = (a, b) => a + b;
```

Fonction anonyme

fonction déclarée sans nom

`this`

`this` est une variable spéciale des fonctions qui pointe vers le propriétaire de la fonction
il est généralement utilisé pour référencer l'objet contenant la méthode où il est utilisé

Callback

le fait de passer une fonction en paramètre d'une autre fonction

```
const scream = function() {
    console.log("AAAAAAAAAH");
}

setTimeout(scream, 5000) // effectue la fonction après 5000 millisecondes
```

comment ça marche ?

```
const repeat = function(callback, times) {
    for(let i = 0; i < times, i++) {
        callback();
    }
}

const scream = function() {
    console.log("AAAAAAAAAH");
}

repeat(scream, 5); // affiche "AAAAAAAAAH" 5 fois
```

possible de passer des variables de la fonction au callback

```
const repeat = function(callback, times) {
    for(let i = 0; i < times, i++) {
        callback(i);
    }
}

repeat((i) = {console.log(i + 2)} , 5); // affiche 2, 3, 4, 5, 6, 7
```

Décomposition et reste

Décomposition

extraire des valeurs d'un tableau

```
const nums = [1, 2, 3];
const [a, b] = nums; // stocke les deux premières valeurs du tableau dans a et b
respectivement
console.log(a); // 1
console.log(b); // 2
```

Reste

`...` : opérateur de reste si à gauche de l'affectation

```
const nums = [1, 2, 3];
const [a, ...moreNums] = nums; // stocke la première valeur du tableau dans a et le reste
dans moreNums(qui est donc un nouveau tableau)
console.log(a); // 1
console.log(moreNums); // [2, 3]
```

on peut donc utiliser `[...copy] = nums` pour créer une copie de l'entièreté du tableau

Spread

`...` : aussi l'opérateur de spread si à droite de l'affectation

```
const n1 = [1, 2, 3];
const n2 = [4, 5, 6];
const combined = [ ... n1,  ... n2]; // combine les tableaux
console.log(combined); // [1, 2, 3, 4, 5, 6]
```

Décomposition d'objets

même principe pour extraire des propriétés spécifiques d'un objet

```
const user = {
    name: "Anna",
    age: 23,
    email: "anna@gmail.com"
}

const { email } = user; // stocke la valeur de la propriété email dans la variable email
console.log(email); // "anna@gmail.com"
```

Fonctions variadiques

fonction qui prend l'opérateur spread en paramètre pour pouvoir lui donner un nombre variable de paramètres

```
const add = ( ... nums) => {
    let sum = 0;
    for (const num of nums) {
        sum += num;
    }
    return sum;
}

const sum1 = add(1, 2, 3); // 6
const sum2 = add(1, 6, 7, 3, 5); // 22
```

Set, Map, Closure

Set

- se crée avec `new Set(...)` et une structure itérable à l'intérieur (tableau, string, …)
- enlève toutes les données dupliquées
- on peut utiliser les boucles `for ... of` ou `forEach`
- `set.size` pour avoir la taille (et non pas `.length`)
- `set.has()` fonctionne comme `array.includes()`
- `set.add()` et `set.delete()` pour ajouter et enlever des valeurs
- pas possible d'aller chercher des valeurs dans un set, principalement pour savoir si il contient un élément donné ou pas

Map

- permet de stocker des paires de clé valeur

- comme un objet mais itérable
- `map.set(key, value)` pour ajouter une paire
- `map.get(key)` pour récupérer une valeur
- ⚠ la boucle `forEach()` avec deux paramètres récupère d'abord `value` puis `key` !!

Enchainement de fonctions

la deuxième fonction s'exécute sur le retour de la première
⇒ faire attention à quels sont les retours des fonctions utilisées !!

Partage de fonctions

partager des fonctions entre des objets similaires par exemple
problème de perdre le `this` utilisé dans des fonctions, besoin de pouvoir le réattribuer

- `function.call()` : prend comme premier paramètre l'objet que `this` appelle, puis les paramètres de la fonction
- `function.apply()` : prend comme premier paramètre l'objet que `this` appelle, puis un tableau des paramètres de la fonction
- `function.bind()` : prend comme seul paramètre l'objet que `this` appelle, puis retourne une nouvelle fonction attachée à cet objet

Closure

retourner une fonction dans une fonction pour protéger des données
quand on retourne une fonction d'une fonction, les variables de la fonction parent restent stockées en mémoire

```
function outer() {
    const secret = "Coucou, je suis la fonction externe";
    function inner() {
        const notSecret = "Coucou, je suis la fonction interne";
        console.log(secret);
        console.log(notSecret);
    }
    return inner;
}

const sayHi = outer(); // sayHi est une variable qui stocke la fonction inner

console.log(sayHi); // affiche le corps de la fonction (donc la fonction inner)
sayHi(); // affiche les deux messages
```

DOM

Document Object Model : représentation structurée des documents HTML
⇒ permet à JavaScript d'accéder et manipuler les éléments

DOM est un API web ⇒ inhérente au navigateur, pas à JavaScript

`document.querySelector("selector")`

- récupère le premier élément correspondant au sélecteur
- même syntaxe que les sélecteurs CSS (`"element"` , `".class"` , `"#id"`)
- retourne sous forme d'objet (avec plein de propriétés)

propriétés intéressantes :

- `textContent` : contenu textuel de l'élément
- `value` : valeur d'un input d'un formulaire
- `style` : un objet reflétant les styles inline de l'élément

`document.querySelectorAll("selector")`

- récupère tous les éléments correspondant au sélecteur
- retourne un tableau d'objets

`classList`

représente une liste des attributs `class` de l'élément

- `.add(class-name)` : ajoute une classe
- `.remove(class-name)` : enlève la classe si elle existe
- `.toggle(class-name)` : ajoute ou enlève la classe selon qu'elle soit déjà dans la liste ou pas

attributs de donnée

un élément HTML a une propriété `dataset` qui contient les attributs de données déclarés dans la balise

- en HTML : attribut de la forme `data-name`
- pour le récupérer en JS, `element.dataset.name`

Evénements

- déclenché par une certaine action
- besoin d'un EventListener pour surveiller le déclenchement de cette action

`element.addEventListener(event, function)` : pour ajouter un listener sur un élément

- `event` : l'événement que le listener surveille (DOM events)
- `function` : la fonction qui s'exécute quand l'event est déclenché

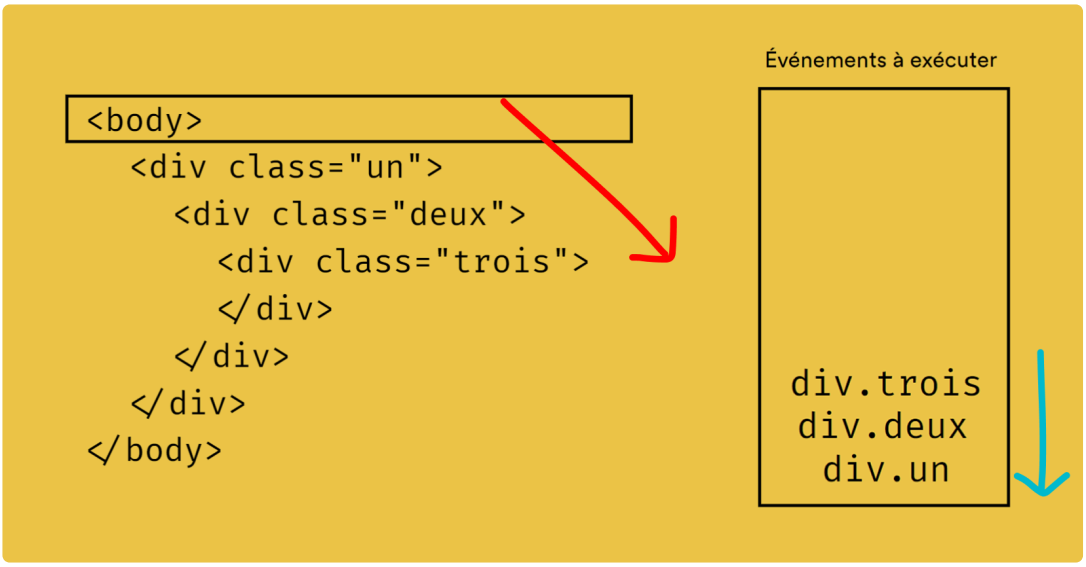
```
// récuperation des éléments
const square = document.querySelector("div");
const counter = document.querySelector("h1");
const button = document.querySelector("button");
const input = document.querySelector("input");

// incrémenter le compteur en cliquant sur un carré
square.addEventListener("click", () => {
```

```
    counter.textContent++;
});

// remplacer par la valeur de l'input au clic du bouton submit
button.addEventListener("click", () => {
    counter.textContent = input.value;
})
```

Propagation d'événements



Capture

navigateur traverse l'arborescence HTML jusqu'à la source du clic
enregistre tous les événements à exécuter

Bubbling

exécute les événements trouvés dans l'ordre inverse

possible de passer un objet d'options à `addEventListener()` :

- `once` : l'écouteur est détruit après avoir invoqué une fois ⇒ on ne peut cliquer qu'une seule fois sur l'objet

Délégation d'événements

ajouter uniquement un listener sur le conteneur pour cibler les éléments contenus dedans plutôt qu'en ajouter un sur chaque élément

`event.currentTarget` retourne l'objet que l'eventListener écoute
`event.target` retourne l'élément le plus proche du curseur au moment de l'event
`[objet HTML].offsetHeight` retourne la hauteur de l'objet (largeur pour `offsetWidth`)
`[objet HTML].tagName` retourne le type de l'objet (div, p, h1, …) parfois en majuscules, parfois pas ⇒ aller checker avec `console.dir(element)`

Asynchronicité

code synchrone : exécuté dans l'ordre, ligne par ligne
code asynchrone : exécuté après qu'une tâches qui tourne en background est terminée
⇒ non-bloquant, le reste du code n'attend pas que la partie asynchrone soit exécutée

AJAX (Asynchronous JavaScript and XML)

- permet de communiquer avec des serveurs distants de manière asynchrone
- appels AJAX pour récupérer dynamiquement des données de serveurs web

API (Application Programming Interface)

- logiciel qui permet d'être utilisé par un autre logiciel ⇒ communication entre les apps
- API "locaux" : DOM API, Geolocation API
- API "distants" : reçoit des requêtes sur un serveur et renvoie les données demandées

`XMLHttpRequest()`

```
const req = new XMLHttpRequest();
req.open("GET", url);
rew.send();

req.addEventListener("load", (e) => {
    e.currentTarget.responseText;
})
```

- permet de faire des requêtes vers un serveur distant sans devoir rafraîchir la page
- attendre la réponse avec un EventListener
- réponse traitée dans le callback

Promises

espace réservé pour le résultat futur d'une opération asynchrone

- opération réussie (fulfilled) ⇒ `.then()`
- opération rejetée (rejected) ⇒ `.catch()`
  ça ne sert à rien de stocker une promesse dans une variable ; stockée avant qu'elle soit terminée donc elle sera de toute manière en attente ⇒ il faut utiliser `.then()`

méthodes sur les promesses

- `.then()` : si l'opération asynchrone réussit ; possible d'en enchaîner plusieurs l'un à la suite de l'autre
- `.catch()` : attrape toute erreur qui à lieu dans la chaîne de promesses ; se place donc tout à la fin des `.then()`
- `.finally()` : s'exécute une fois que toute la chaîne de promesses a été résolue

`fetch()`

alternative moderne à `XMLHttpRequest()`, prend l'url en paramètre ⇒ `fetch(url)`

- retourne systématiquement une promesse
- seul motif de rejet est une erreur de réseau
- données retournées avec `fetch()` sous forme de texte ⇒ utiliser la méthode `.json()` sur la promesse
- `.json()` retourne aussi une promesse, donc il faut utiliser un `.then` pour extraire les données

```
fetch(url)
.then(res => res.json())
.then(data => console.log(data))
```

- si on fait un autre `fetch()` (ou une autre requête asynchrone) dans le `.then()`, retourner la promesse et faire le nouveau `.then` à la suite des autres

`Promise.all()`

plusieurs `fetch()` en parallèle

```
Promise.all([fetch(url), fetch(url), ...])
.then(responses => Promise.all(responses.map(response => response.json())))
.then(data => console.log(data))
```

- fulfilled si toutes les promesses le sont
- rejected dès qu'une promesse l'est
- valeur de retour : tableau de toutes les résolutions

Event loop

dans le moteur javascript :

- heap : où sont stockés les objets
- call stack : pile des actions à faire ⇒ ordre

déroulement :

- tâches synchrones exécutées immédiatement, tâches asynchrones mises en arrière-plan
- quand une tâche asynchrone est terminée, celles qui dépendent d'elle (EventListeners, `.then()`, ...) sont déplacées dans la **callback queue**
- si le call stack est vide, la première tâche de la callback queue est déplacée dans le call stack et exécutée
- déplacement de la callback queue au call stack ⇒ Event Loop

comme il faut que le call stack soit vide pour effectuer les callbacks, ça peut ralentir leur exécution

async/await

façon d'écrire du code asynchrone (suite de `.then`) de manière synchrone

> ⓘ **Sucre syntaxique**

abstraction du fonctionnement de JavaScript
n'apporte rien au fonctionnement, uniquement une manière d'écrire

`async`

mot-clé placé devant la déclaration d'une fonction qui fait qu'elle retourne systématiquement une promesse
→ la valeur de résolution de la promesse est la valeur retournée par la fonction

```
const add = async (a, b) => {
    return a + b;
};

console.log(add(2,2)); // Promise
add(2,2).then(result => console.log(result)); // 4
```

`await`

ne peut être utilisé qu'à l'intérieur d'une fonction async
pause l'exécution de la fonction jusqu'à que la promesse associée soit résolue
⇒ permet de stocker la valeur de résolution dans une variable
gestion des erreurs avec un try/catch

```
async function getCountries(name) {
try {
    const res = await fetch(`https://restcountries.com/v3.1/name/${name}`);
    const countries = await res.json();
    if(countries[0]) {
        console.log(countries);
    } else {
        throw new Error("No countries found");
}
catch(e) {
    console.log(e);
}
}
```

Comparaison

XMLHttpRequest

```
function getCountries(name) {
    const request = new XMLHttpRequest();
    request.open("GET", `https://restcountries.com/v3.1/name/${name}`);
    request.send();

    request.addEventListener("load", () => {
        const countries = JSON.parse(request.response);
        console.log(countries);
```

```
    })
}
```

Promise

```
function getCountries(name) {
    fetch(`https://restcountries.com/v3.1/name/${name}`)
    .then(res => res.json())
    .then(data => console.log(data));
}
```

Async/await

```
async function getCountries(name) {
    const res = await fetch(`https://restcountries.com/v3.1/name/${name}`);
    const countries = await res.json();
    console.log(countries);
}
```

Prototypes

en POO classique, les objets sont des instances d'une classe
en JS, les objets sont liés à un prototype

- le prototype contient les méthodes
- l'objet accède aux méthodes

3 manières de créer des prototypes/objets :

- Constructor functions
- Classes ES6
- `Object.create()`

Constructor functions

fonction normale appelée avec le mot-clé `new`

- crée un nouvel objet
- la fonction constructor est appelée (`this` se réfère à l'objet)
- l'objet est automatiquement lié à un prototype
- la fonction retourne l'objet (qui peut donc être stocké dans une variable)

```
const Person = function (name, year) {
    this.name = name;
    this.year = year;
};
```

```
const anna = new Person("Anna", 1985);
```

`Person.prototype` est le prototype des objets créés avec la fonction `Person()`

on peut ajouter des méthodes au prototype avec `Person.prototype` :

```
Person.prototype.calcAge = function () {
    const currentYear = new Date.getFullYear();
    const age = currentYear - this.year;
}
```

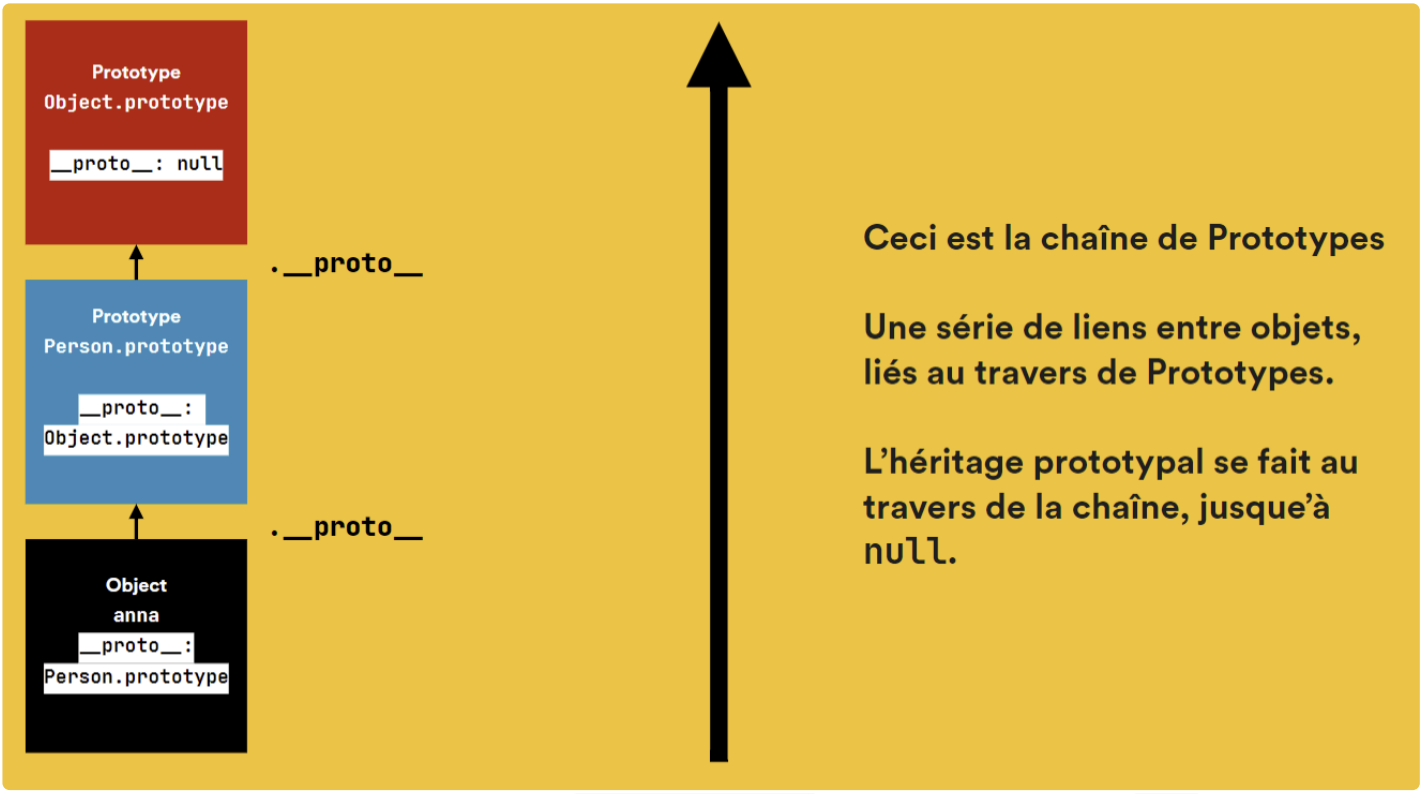
Héritage prototypal

l'objet `anna` ne contient pas lui-même la fonction `calcAge()` :

```
anna = {
    name: "Anna",
    year: 1985,
    __proto__: Person.prototype
}
```

quand on appelle `calcAge()` sur `anna`, JavaScript va regarder si l'objet contient la fonction ; sinon, il va remonter au prototype grâce à `.__proto__` et vérifier si celui-ci contient la fonction ⇒ Héritage prototypal

Chaîne de prototypes



tous les objets en JS finissent par hériter de `Object.prototype`, qui lui-même hérite de `null`

Classes

- sucre syntaxique pour travailler avec des classes, fonctionne bien avec les prototypes
- méthode `constructor()` est l'équivalent de la constructor function
- méthodes déclarées dans le bloc de classe
- nouvelle instance aussi crée avec le mot-clé `new`

sans les classes :

```
const Person = function (name, year) {
    this.name = name;
    this.year = year;
};

Person.prototype.calcAge = function () {
    const currentYear = new Date.getFullYear();
    return age = currentYear - this.year;
}

const anna = new Person("Anna", 1985);
```

avec les classes :

```
class Person {
    constructor(name, year) {
        this.name = name;
        this.year = year;
    }

    calcAge() {
        const currentYear = new Date.getFullYear();
        return age = currentYear - this.year;
    }
}

const anna = new Person("Anna",1985)
```

- classes ne peuvent pas être hoisted
- classes automatiquement exécutées en strict mode

Héritage

mot-clé `extends` rend le lien entre les prototypes explicites :

```
class Student extends Person { ... }
```

la fonction `super()` se réfère à la fonction `constructor()` de la classe parent

Getter + Setter

setter + getter permettent d'utiliser des valeurs calculées comme si c'était des propriétés
getter a forcément aucun paramètre et retourne une valeur
setter prend forcément un paramètre

faire `this.name = name` appelle le setter s'il existe
⇒ attention à ne pas rappeler le setter dans le setter !!

Encapsulation

encapsulation : pas pour empêcher la console/utilisateur de voir des infos, plutôt pour empêcher je sais plus

propriétés/méthodes privées

- pour empêcher de modifier les propriétés directement dans le code
- n'empêche pas d'y accéder ou de les modifier depuis la console
- déclarée avec un `#` devant : `#privateProperty` vs `publicProperty`

Modules

bout de code réutilisable qui encapsule des détails d'implémentation

- en javascript : 1 module = 1 fichier
- il faut préciser qu'un fichier est un module dans le HTML : `<script src="app.js" type="module">`
- import/export uniquement possible dans un module
  - import d'éléments (classe, fonction, variable) d'autres modules
  - export d'éléments du module
- variables d'un module définies uniquement dans le module
- strict mode par défaut
- dans un module, `this` est undefined
- le chargement des modules est asynchrone

avantages

- on peut assembler des modules simples pour composer des applis complexes
- modules développés en isolation les uns des autres
- code de bas-niveau dans un module et import de ces abstractions dans d'autres
- codebase plus organisée
- on peut réutiliser des bouts de code dans des projets différents

export

- un élément doit être exporté pour pouvoir être importé dans un autre module
- export d'un élément avec un nom ⇒ export nommé
- pattern classique : 1 module = 1 élément ⇒ export par défaut

```
export class Car { ... } // exporte l'élément
export { Car, Bus } // exporte tous les éléments indiqués
```

```
export default class { ... } // export par défaut
```

import

- le contenu du module importé est exécuté avant le contenu du module

```
import "./Car.js" // importe tout le module
import { Car, Bus } from "./Car.js" // importe les éléments indiqués du module
import * as Transport from "./Car.js" // importe tous les éléments du module comme un objet qui contient les éléments
import Voiture from "./Car.js" // import par défaut, on lui donne un nom
```

node.js

runtime de javascript ⇒ pour exécuter du code en dehors du navigateur

npm

gestionnaire de paquets pour javascript de node
(brew pour mac, apt pour linux)
permet d'installer des paquets (ensemble de fichiers javascript, aussi appelé library)
ajoute des éléments :

- `package.json` : fichier de gestion des dépendances (paquets installés desquels dépend le code)
- `package-lock.json` : fichier de gestion des dépendances des dépendances
- `node_modules` : dossier contenant tous les paquets nécessaires
  Attention à ne jamais commit le dossier `node_modules`, il est très très lourd !!

```
npm init # initialise npm
npm install # installe un paquet (l'ajoute à package.json)
```

versioning

version composée de 3 parties (par ex `4.17.22`)

- version majeure : changements assez importants pour casser le code
- version mineure : ajout de nouvelles fonctionnalités, mais ne casse pas le code
- patch : tout petit changement, style correction de bugs

npm met à jour les dépendances automatiquement
→ normalement bloqué sur la version majeure pour ne pas casser le code
→ représenté avec un circonflexe : `^4.17.22` signifie que la version majeure est bloquée sur 4

`npx`

mot-clé pour exécuter une commande qui se trouve dans le dossier `node_modules`

npm scripts

scripts définis dans `package.json` exécutables avec `npm run [script]` (deux exceptions, `npm test` et `npm start` n'ont pas besoin de `run`)
comme on utilise npm pour lancer le script, on n'a pas besoin de spécifier `npx` dedans

bundler

outil de build :

- regroupe les modules installés (= bundling)
- crée un unique fichier `.js` avec tout le code des différents fichiers et dépendances
- fait aussi le transpiling et polyfilling
- par exemple, vite

transpiling

minification : tous les espaces, sauts à la ligne, caractères superflus sont enlevés

polyfilling

- rend le code compatible avec des versions plus anciennes des navigateurs (des fonctionnalités du JS moderne peuvent ne pas être supportées)
- le bundler ajoute du code (= polyfill) pour réimplémenter ces fonctionnalités de sorte à ce qu'elles soient comprises par ces navigateurs
- plus on veut être compatible avec des vieilles versions, plus on doit ajouter de polyfill, plus le fichier est grand

avec vite, on peut spécifier dans `vite.config.js` la plus ancienne version des navigateurs compatibles

Vite

- outil de build (bundling + transpiling, polyfilling)
- fait partie de l'écosystème node ⇒ peut s'installer avec npm
- dev dependency ⇒ s'installe avec `npm install -D vite`

dev dependencies

un module qui n'est pas utilisé directement dans le code, mais nécessaire au développement
il faut l'installer avec `npm install -D` pour qu'il soit considéré comme dev dependency